

Process Calculi Formalized in Lean 4:

Rho, Pi, MQ, and Modal Mu-Calculus with Cross-Calculus Bridges

Zar

Oruži*

March 2026

Abstract

We present a comprehensive Lean 4 formalization of five process-algebraic systems—the ρ -calculus, π -calculus, MQ-calculus, MeTTa-calculus, and modal μ -calculus—together with three cross-calculus bridges: a π -to- ρ encoding with forward simulation and backward admin reflection, an MQ-to-MORK interoperability bridge, and a MeTTa-to- ρ shared-fragment translation with open-map self-bisimulation witness and forward simulation. The formalization totals over 17,500 lines of Lean 4.28 across 55 files with zero `sorry` and zero custom axioms. Shared infrastructure—typeclasses for parallel composition, restriction, structural congruence, and reflexive-transitive closure—eliminates approximately 40% of cross-calculus duplication. Quantale-valued semantics for μ -calculus enables instantiation to probabilistic, fuzzy, tropical, and PLN-evidence domains.

1 Introduction

Process calculi provide compositional models of concurrent, communicating systems. In this paper we report on the formal verification, in the Lean 4 proof assistant [2], of five such calculi and the bridges that connect them.

1. The **ρ -calculus** [9]: a reflective higher-order calculus where names *are* quoted processes, supporting structural congruence, a COMM reduction rule, an executable engine with soundness proof, the Spice calculus [8] for n -step lookahead, and derived replication and restriction.
2. The **π -calculus** [11, 12]: the standard calculus of mobile processes, with a full encoding into the ρ -calculus following Lybech’s name-server approach [6], including forward simulation, encoding morphism structure, weak bisimilarity, and backward admin reflection.
3. The **MQ-calculus** [13]: a quantum process calculus where communication *is* measurement, producing binary outcomes governed by Born-rule probabilities. We formalize backend-parametric denotational semantics with a statevector instantiation and an interoperability bridge to MORK [10].
4. The **MeTTa-calculus** [7]: a symmetric reflective higher-order concurrent calculus with backchaining, formalizing the core of the MeTTa language with COMM and REFL rules, an executable reduction engine, a premise-IR adequacy bridge, and a shared-fragment translation into the ρ -calculus with open-map self-bisimulation witness.

*“Oruži” denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI).

Component	Location	Files	Lines
ρ -calculus	ProcessCalculi/RhoCalculus/	11	4,954
π -calculus	ProcessCalculi/PiCalculus/	19	9,134
MQ-calculus	ProcessCalculi/MQCalculus/	10	1,433
MeTTa-calculus	ProcessCalculi/MeTTaCalculus/	9	1,174
Common infra	ProcessCalculi/Common/	4	246
Modal μ -calc	Logic/ModalMuCalculus.lean	1	360
Quantale sem.	Logic/ModalQuantaleSemantics.lean	1	520
Grand total		55	17,821

Table 1: Formalization summary: 0 sorries, 0 custom axioms throughout.

5. The **modal μ -calculus** [5, 1]: propositional modal logic with least and greatest fixed points. We provide both standard Boolean semantics and quantale-valued semantics, and prove that the Galois connection in ρ -calculus *is* diamond-box duality.

Three bridges tie these together: the π -to- ρ encoding with simulation theorems, an MQ-MORK binary-fold bridge, and a MeTTa-to- ρ shared-fragment translation. All proofs are checked by the Lean 4.28 kernel with zero `sorry` [14].

Table 1 gives line counts; total formalization exceeds 17,500 lines.

2 The Rho-Calculus

The ρ -calculus [9] is a reflective higher-order process calculus in which names are quoted processes: every name x is of the form $@P$ for some process P . This “reflection” dissolves the traditional name/process distinction and raises expressiveness questions beyond purely behavioral formalisms (Section 6). The formalization comprises 11 files totaling 4,954 lines under `ProcessCalculi/RhoCalculus/`.

2.1 Syntax

Process syntax uses a locally nameless representation. The core inductive type `Pattern` (in `Types.lean`, 221 lines) includes:

Listing 1: Rho-calculus process syntax (from `Types.lean`)

```

inductive Pattern : Type where
  | pNil      : Pattern
  | pPar      : Pattern -> Pattern -> Pattern
  | pInput    : Pattern -> Pattern -> Pattern -> Pattern
  | pOutput   : Pattern -> Pattern -> Pattern
  | pDrop     : Pattern -> Pattern
  | pQuote    : Pattern -> Pattern
  | pVar      : Nat -> Pattern
  deriving DecidableEq, Repr

```

The `pQuote/pDrop` pair implements reflection: quoting a process produces a name, and dropping a name recovers the process.

2.2 Structural Congruence

Structural congruence (`StructuralCongruence.lean`, 287 lines) is formalized as an inductive relation including alpha-equivalence, parallel commutativity and associativity, flattening of nested parallels, and the property that quoting respects structural congruence:

Theorem 2.1 (Quote Respects SC). *If $P \equiv_{\text{SC}} Q$ then $@P \equiv_{\text{SC}} @Q$.*

2.3 COMM Rule and Reduction

The COMM rule (`CommRule.lean`, 298 lines) formalizes the core communication step: when an output $\bar{x}(y)$ meets an input $x(z).P$, the name y is substituted for z in P . The full reduction relation (`Reduction.lean`, 430 lines) closes COMM under parallel contexts and structural congruence.

2.4 Executable Engine and Soundness

An executable reduction engine (`Engine.lean`, 427 lines) provides a fuel-bounded decision procedure for single-step reduction. A separate soundness proof (`Soundness.lean`, 739 lines) shows that every engine-computed reduction is derivable in the inductive relation:

Theorem 2.2 (Engine Soundness). *If $\text{engineStep fuel } p = \text{some } q$ then $\text{Reduces } p \ q$.*

Remark 2.3 (COMM and Predecessor-Finiteness). The COMM rule erases the channel name: the variable n binding the communication channel appears in the LHS but not in the RHS. Consequently, for any fixed target q' , infinitely many distinct predecessors exist (one per choice of n), and the ρ -calculus reduction is not predecessor-finite (`not_global_hPredFinite_langReduces_rhoCalc`). A companion formalization [15] identifies a decidable structural sufficient condition (`LangPredFiniteSafe`) under which predecessor-finiteness holds; the ρ -calculus provably fails it (`rhoCalc_not_langPredFiniteSafe`). This is the precise structural reason that global Hennessy–Milner equivalence requires additional quotienting for the ρ -calculus.

2.5 Spice Calculus: Present Moment and Lookahead

The Spice calculus [8] extends ρ -calculus COMM with n -step lookahead (`SpiceRule.lean`, 419 lines; `PresentMoment.lean`, 923 lines). Agents receive the set of states reachable in $\leq n$ steps before committing to an interaction:

$$\text{futureStates}(p, n) = \{q \mid p \rightsquigarrow^n q\}$$

When $n = 0$ this recovers standard ρ -calculus COMM. The “present moment” ($n = 1$) gives agents awareness of their immediate interaction possibilities. Key theorems include compositional n -step parallel congruence lifting (`ReducesN.par_head`, `ReducesN.par_any_pos`) and `futureStates_par_head`.

2.6 Derived Replication and Restriction

The file `DerivedRepNu.lean` (669 lines) derives replication $!P$ and restriction $(\nu x) P$ as definable constructs within the ρ -calculus, proving they satisfy the expected reduction and congruence laws.

3 The Pi-Calculus and the Pi-to-Rho Encoding

The π -calculus [11, 12] is formalized in 19 files totaling 8,967 lines under `ProcessCalculi/PiCalculus/`. We follow the Lybech 2022 encoding [6] of π -calculus into ρ -calculus via a name-server approach.

3.1 Syntax and Reduction

The π -calculus syntax (`Syntax.lean`, 85 lines) and reduction relation (`Reduction.lean`, 405 lines) follow standard presentations [12].

3.2 Lybech 2022 Encoding

The encoding (`RhoEncoding.lean`, 375 lines) maps π -calculus names to ρ -calculus names via a name server that mediates between the flat name space of π -calculus and the reflective name space of ρ -calculus.

3.3 Forward Simulation

The key correctness result is forward simulation (`ForwardSimulation.lean`, 1,387 lines): if a π -calculus process reduces, its encoding reduces (modulo administrative steps) in ρ -calculus.

Theorem 3.1 (Forward Simulation). *If $P \rightarrow_\pi P'$ then $\llbracket P \rrbracket \rightarrow_\rho^* \llbracket P' \rrbracket$ (up to administrative reductions).*

The theorem name in Lean is `encoding_forward_single_step_rf`.

3.4 Encoding Morphism Structure

The file `EncodingMorphism.lean` (894 lines) establishes that the encoding preserves the algebraic structure of parallel composition, restriction, and nil:

$$\llbracket P \mid Q \rrbracket \equiv \llbracket P \rrbracket \mid \llbracket Q \rrbracket, \quad \llbracket 0 \rrbracket \equiv 0, \quad \llbracket (\nu a) P \rrbracket \equiv (\nu \hat{a}) \llbracket P \rrbracket$$

3.5 Weak Bisimilarity

Weak bisimilarity (`WeakBisim.lean`, 230 lines; `WeakBisimDerived.lean`, 149 lines) formalizes the standard notion from [12]: two processes are weakly bisimilar when every observable reduction step of one can be matched (up to silent τ -steps) by the other, and vice versa. The definition is Prop-valued and coinductive-style: a witness is an explicit relation R that is symmetric, closed under forward and backward lifting, and preserves barbed observability restricted to a finite name set N (following Lybech’s N -restricted barbed bisimilarity [6]). `WeakBisimOpenMapBridge.lean` connects this to the open-maps approach of Joyal, Nielsen, and Winskel [4] by showing that weak N -restricted bisimilarity coincides with path bisimilarity over a `BisimulationKit` that instantiates the generalized open-map framework from `CategoryTheory/GeneralizedOpenMaps.lean`.

3.6 Backward Admin Reflection

The largest single file in the formalization, `BackwardAdminReflection.lean` (2,745 lines), proves that administrative reductions introduced by the encoding can be reflected backward: if $\llbracket P \rrbracket \rightarrow_\rho^* Q$ via admin steps, then Q is structurally congruent to some $\llbracket P' \rrbracket$. The file now also exposes a fully general backward-simulation endpoint over encoded step-sources: `weak_backward_full_simulation_encodedSC`, together with encoded-source specializations `weak_backward_full_simulation_encode` and `weak_backward_full_simulation_reflect`. These theorems flatten the reflected trace decomposition into three explicit branches: reflexive endpoint, admin-progress branch, or reflected π -step branch with head/tail trace data. This completes the backward direction of the simulation argument: every administrative reduction sequence starting from an encoded process eventually reaches an encoded image again.

4 The MQ-Calculus

The MQ-calculus [13] unifies communication and quantum measurement: a COMM event produces a binary measurement outcome governed by Born-rule probabilities. The formalization (10 files, 1,433 lines) covers syntax, operational reduction, backend-parametric denotational semantics, and a cross-calculus bridge to MORK. This extends the communicating quantum processes framework of Gay and Nagarajan [3] with explicit measurement branching in the reduction semantics.

4.1 Syntax

The process grammar uses de Bruijn-style wire indexing and typed gate specifications:

Listing 2: MQ-calculus process syntax (from `Syntax.lean`)

```
inductive GateOp : Type where
  | H | X | Z
  | custom (name : String)
  deriving DecidableEq, Repr

structure GateSpec where
  op : GateOp
  target : Nat
  deriving DecidableEq, Repr

inductive Process : Type where
  | MQNil : Process
  | MQPar : Process -> Process -> Process
  | MQNu : Process -> Process
  | MQGate : GateSpec -> Process -> Process
  | MQOut : Nat -> Process
  | MQIn : Nat -> Process -> Process -> Process
  deriving DecidableEq, Repr
```

The `MQIn` constructor takes two continuations—one for each measurement outcome—making measurement branching explicit in the syntax.

4.2 Communication as Measurement: The COMM Rule

The COMM rule (`CommRule.lean`, 87 lines) produces a binary measurement outcome. A central theorem provides a constructive both-outcomes witness:

Listing 3: COMM rule with constructive both-branches (from `CommRule.lean`)

```
inductive Outcome : Type where
  | zero : Outcome
  | one : Outcome
  deriving DecidableEq

inductive CommReduction :
  Nat -> Process -> Process -> MeasurementBranch -> Prop where
  | outcome_zero (i : Nat) (p q : Process) :
    CommReduction i p q { outcome := .zero, result := p }
  | outcome_one (i : Nat) (p q : Process) :
    CommReduction i p q { outcome := .one, result := q }
```

```

theorem comm_both_outcomes (i : Nat) (p q : Process) :
  CommReduction i p q { outcome := .zero, result := p }
  /\ CommReduction i p q { outcome := .one, result := q } :=
  (.outcome_zero i p q, .outcome_one i p q)

```

Born-style probabilities are defined separately on quantum state vectors; the branch relation itself remains structural and proof-friendly.

4.3 Backend-Parametric Denotational Semantics

The semantics (`Backend.lean`, 380 lines; `Denotational.lean`, 229 lines) is parameterized by a backend interface that provides gate action, fresh allocation, branch probabilities, and post-measurement collapse:

Listing 4: Backend interface (from `Backend.lean`)

```

structure MQSemanticsBackend where
  branchProb      : Nat -> Outcome -> QState -> Real
  branchProb_nonneg : forall i out st, 0 <= branchProb i out st
  branchProb_sum_one : forall i st,
    branchProb i .zero st + branchProb i .one st = 1
  applyGate       : GateSpec -> QState -> QState
  allocFresh      : QState -> QState
  collapse        : Nat -> Outcome -> QState -> QState

```

The current `statevectorBackend` instantiation gives executable semantics while preserving theorem-level invariants. A key normalization result ensures physical consistency:

Theorem 4.1 (Post-Collapse Normalization). *collapseByOutcome_norm_eq_one_of_raw_pos: if $0 < \text{rawOutcomeProb}(i, \text{out}, st)$ then $\|\psi'\| = 1$ after collapse.*

4.4 MQ-MORK Bridge

MQ measurement branching is linked to MORK binary-fold branching (`Interoperability.lean`, 37 lines) via:

Theorem 4.2 (MQ-MORK Interoperability). *comm_nondeterminism_iff_mork_binary: the two COMM outcomes of a measurement correspond bijectively to the two sub-results of a MORK binary fold step.*

This ties communication-level nondeterminism in MQ directly into the MORK three-phase execution model [10].

Executable witness. Two MM2 scripts exercise the bridge concretely on the MORK runtime. MORK operates on a fact database: an `exec` rule matches facts against a pattern (after the comma) and rewrites them via a template ((+ ...) adds a fact, (- ...) removes one). Both scripts set up identical initial facts and an identical fold rule, differing only in which branch is selected.

Script 1 (`binary_fold_pick0.mm2`)—selects branch 0:

Listing 5: Pick outcome 0 (from `binary_fold_pick0.mm2`)

```

; Initial facts: a pending query with two candidate sub-results
(wait q1)
(subResult q1 0 P)

```

```

(subResult q1 1 Q)

; Choose branch 0 in this run
(pick q1 0)

; Fold rule: when (wait $q), (pick $q $i), and (subResult $q $i $r)
; all match, emit (commOutcome $q $i $r) and consume the wait token
(exec (0 fold)
  (, (wait $q)
    (pick $q $i)
    (subResult $q $i $r))
  (0 (+ (commOutcome $q $i $r))
    (- (wait $q))))

```

The pattern binds $\$q=q1$, $\$i=0$, $\$r=P$ (since `(pick q1 0)` selects index 0). The template emits `(commOutcome q1 0 P)`.

Script 2 (`binary_fold_pick1.mm2`)—selects branch 1:

Listing 6: Pick outcome 1 (from `binary_fold_pick1.mm2`)

```

; Same initial facts
(wait q1)
(subResult q1 0 P)
(subResult q1 1 Q)

; Choose branch 1 in this run
(pick q1 1)

; Same fold rule
(exec (0 fold)
  (, (wait $q)
    (pick $q $i)
    (subResult $q $i $r))
  (0 (+ (commOutcome $q $i $r))
    (- (wait $q))))

```

Now $\$i=1$ and $\$r=Q$, so the template emits `(commOutcome q1 1 Q)`.

The two outputs—`(commOutcome q1 0 P)` and `(commOutcome q1 1 Q)`—are the operational witnesses for `mork_mq_nondeterminism_corresponds`: the *same* fold rule, applied to the *same* query with the *same* two candidate sub-results, produces *either* outcome depending solely on the branch selection. This is precisely the structural binary nondeterminism that `comm_nondeterminism_iff_mork_binary` formalizes in Lean: MQ’s two `CommReduction` outcomes ($\text{zero} \rightarrow P$, $\text{one} \rightarrow Q$) biject with MORK’s two `FoldPicksSubResult` choices (`subResult0`, `subResult1`).

5 The MeTTa-Calculus

The MeTTa-calculus [7] formalizes the core of the MeTTa language as a symmetric reflective higher-order concurrent calculus with backchaining. The formalization comprises 9 files totaling 1,174 lines under `ProcessCalculi/MeTTaCalculus/`.

5.1 Syntax

The process grammar is:

$$P, Q ::= \mathbf{0} \mid P \mid Q \mid \text{for}(t \leftarrow x) P \mid x ? P \mid *x$$

$$x, y ::= @(P) \quad t, u ::= (P) \mid \text{true} \mid \text{false} \mid n \mid s \mid \text{sym}$$

where $\text{for}(t \leftarrow x) P$ is guarded input (listen for pattern t on channel x , then continue as P), $x ? P$ is reflection (inspect P and emit its future state on x), $@(P)$ quotes a process into a name, and $*x$ drops (unquotes) a name back to a process.

`Syntax.lean` (135 lines) encodes the grammar as abbreviations of the shared `Pattern` AST:

Listing 7: MeTTa-calculus syntax (from `Syntax.lean`)

```
abbrev Proc := Pattern    -- processes
abbrev Name := Pattern    -- names (quoted processes)
abbrev Term := Pattern    -- terms (data values)

def pZero    : Proc := .apply "MZero" []
def pPar     (elems : List Proc) : Proc :=
  .collection .hashBag elems none
def nQuote   (p : Proc) : Name := .apply "MQuote" [p]
def pDrop    (x : Name) : Proc := .apply "MDrop" [x]
def pFor     (t : Term) (x : Name) (k : Proc) : Proc :=
  .apply "MFor" [t, x, k]
def pReflect (x : Name) (p : Proc) : Proc :=
  .apply "MRef" [x, p]
```

Two language definitions are provided: `mettaCalc` (full COMM + REFL rules) and `mettaCalcCommOnly` (the COMM-only fragment, used to avoid self-reference in the REFL rule's recursive stepping).

5.2 Structural Congruence

Structural congruence ($\equiv$) is the smallest equivalence closed under:

$$\begin{aligned} P \mid \mathbf{0} &\equiv P && (\text{NIL}) \\ P \mid Q &\equiv Q \mid P && (\text{COMM}) \\ (P \mid Q) \mid R &\equiv P \mid (Q \mid R) && (\text{ASSOC}) \\ @(*x) &\equiv x && (\text{QUOTEDROP}) \end{aligned}$$

`StructuralCongruence.lean` (67 lines) defines the inductive relation `SC` (notation $\approx_m$) with 11 constructors covering all clauses above plus α -equivalence, permutation, and pointwise congruence:

Listing 8: Structural congruence (from `StructuralCongruence.lean`)

```
inductive SC : Proc -> Proc -> Prop where
| refl    (p : Proc) : SC p p
| symm    (p q : Proc) : SC p q -> SC q p
| trans   (p q r : Proc) : SC p q -> SC q r -> SC p r
| alpha   (p q : Proc) : p = q -> SC p q
| par_nil_left (p : Proc) :
  SC (pPar [pZero, p]) p
| par_nil_right (p : Proc) :
```



```

    SC (pPar [p, pZero]) p
| par_comm (p q : Proc) :
    SC (pPar [p, q]) (pPar [q, p])
| par_assoc (p q r : Proc) :
    SC (pPar [pPar [p,q], r]) (pPar [p, pPar [q,r]])
| par_perm (xs ys : List Proc) :
    xs.Perm ys -> SC (pPar xs) (pPar ys)
| par_cong (xs ys : List Proc) :
    xs.length = ys.length ->
    (forall i hx hy, SC (xs.get {val:=i,isLt:=hx})
                        (ys.get {val:=i,isLt:=hy})) ->
    SC (pPar xs) (pPar ys)

```

Paper-mapped equivalence theorems:

- `paper_equiv_par_nil_left`: $P \mid \mathbf{0} \approx_m P$
- `paper_equiv_par_nil_right`: $\mathbf{0} \mid P \approx_m P$
- `paper_equiv_par_comm`: $P \mid Q \approx_m Q \mid P$
- `paper_equiv_par_assoc`: $(P \mid Q) \mid R \approx_m P \mid (Q \mid R)$

5.3 Reduction Semantics

The calculus has two reduction rules. The COMM rule performs a symmetric rendezvous: two guarded inputs listening on the same channel unify their payload patterns and continue with substituted bodies:

$$\frac{\sigma = mgu(t, u)}{\text{for}(t \leftarrow x) P \mid \text{for}(u \leftarrow x) Q \longrightarrow P\dot{\sigma} \mid Q\dot{\sigma}} \quad (\text{COMM})$$

where $\dot{\sigma}$ is *dot-substitution*: each binding $(v \mapsto T)$ in σ becomes $(v \mapsto @ (T))$, quoting the term into a name. The REFL rule lets a process inspect its own future via one-step COMM-only lookahead:

$$\frac{P \longrightarrow_{\text{COMM}} P'}{x ? P \longrightarrow \text{for}((P') \leftarrow x) \mathbf{0}} \quad (\text{REFL})$$

`Reduction.lean` (264 lines) encodes both rules as OSLF rewrite rules with premise queries. The COMM rule uses a fuel-bounded first-order unifier (256 steps) with an occurs check:

Listing 9: COMM rewrite rule (from `Syntax.lean`)

```

private def commSymRule : RewriteRule := {
  name := "CommSym"
  typeContext := [ ("t", .base "Term"), ("u", .base "Term"),
    ("x", .base "Name"), ("p", .base "Proc"),
    ("q", .base "Proc"), ("pOut", .base "Proc"),
    ("qOut", .base "Proc")]
  premises := [
    .relationQuery relMettaComm
    [.fvar "t", .fvar "u", .fvar "p", .fvar "q",
     .fvar "pOut", .fvar "qOut"] ]
  left := .collection .hashBag [
    .apply "MFor" [.fvar "t", .fvar "x", .fvar "p"],
    .apply "MFor" [.fvar "u", .fvar "x", .fvar "q"]
  ]
}

```

```

] (some "rest")
right := .collection .hashBag
  [.fvar "pOut", .fvar "qOut"] (some "rest")
}

```

Listing 10: REFL rewrite rule (from `Syntax.lean`)

```

private def reflRule : RewriteRule := {
  name := "Ref1"
  typeContext := [("x", .base "Name"),
    ("p", .base "Proc"), ("pNext", .base "Proc")]
  premises := [.relationQuery relMettaStepNoReflect
    [.fvar "p", .fvar "pNext"]]
  left := .apply "MRef" [.fvar "x", .fvar "p"]
  right := .apply "MFor"
    [.apply "MTermProc" [.fvar "pNext"],
      .fvar "x", .apply "MZero" []]
}

```

The dot-substitution and unification implementations complete the executable semantics:

Listing 11: Dot-substitution and step function (from `Reduction.lean`)

```

def dotBindings (s : Bindings) : Bindings :=
  s.map (fun (x, v) => (x, nQuote v))

def applyDot (s : Bindings) (p : Proc) : Proc :=
  applyBindings (dotBindings s) p

def step (p : Proc) : List Proc :=
  rewriteWithContextWithPremisesUsing
    mettaCalcRelEnv mettaCalc p

```

Positive and negative canaries validate both rules at the kernel level:

- `paper_comm_positive/paper_comm_negative`: a concrete COMM rendezvous succeeds; mismatched channels block.
- `paper_refl_positive/paper_refl_negative`: a concrete REFL step emits a future state; a non-stepping body produces no output.

5.4 Premise Intermediate Representation and Adequacy

`Premises.lean` (88 lines) declares a `PremiseProgram`—a datalog-style intermediate representation (IR) with two relations and two builtins:

$$\text{mettaComm}(t, u, P, Q, P', Q') \quad \text{mettaStepNoReflect}(\text{src}, \text{tgt})$$

$$\text{mettaCommWitness}(t, u, P, Q) \rightsquigarrow \text{MRef}(P', Q') \quad \text{mettaCommOnlyStep}(\text{src}) \rightsquigarrow \text{tgt}$$

with machine-checked stratification and wellformedness proofs.

`Adequacy.lean` (79 lines) bridges the abstract premise contract to the concrete runtime:

Theorem 5.1 (Premise–Runtime Adequacy). *The following are proven:*

1. *runtime_covers_declared_relations*: every declared relation is implemented by the runtime dispatcher.
2. *runtime_covers_declared_builtins*: every declared builtin is implemented.
3. *stepViaPremiseContract_eq_step*: the IR-based stepping function agrees with the direct step function.
4. *reducesViaPremiseContract_iff_reduces*: $\text{ReducesViaPremiseContract } p \ q \iff \text{Reduces } p \ q$.

5.5 Interoperability with ρ -Calculus

`Interoperability.lean` (301 lines) provides a shared-fragment translation from the quote/drop/-parallel subset of MeTTa-calculus into ρ -calculus. The translation `toRhoSharedProc?` succeeds on the translatable fragment and fails gracefully outside it.

For any successfully translated process, the file proves path-bisimulation self-witnesses, intrinsic rho-side inversion, and shared-core forward/backward simulation:

- `shared_translation_pathBisim_self`: the translated image is path-bisimilar to itself under the ρ -calculus `BisimulationKit`.
- `shared_translation_esBisim_self`: the corresponding (E, S) -bisimilarity corollary.
- `rhoSharedCore_step_inversion`: intrinsic inversion of `RhoSharedCoreReduces`.
- `rhoSharedCore_stepStar_inversion`: intrinsic inversion of `RhoSharedCoreReducesStar`.
- `sharedCore_step_forward_restricted`: one-step forward simulation on the quote/drop/-parallel shared-core fragment.
- `sharedCore_step_backward_restricted`: source-anchored one-step backward simulation.
- `sharedCore_step_bisimulation`: combined one-step forward+backward shared-core bisimulation correspondence.
- `sharedCore_stepStar_forward_restricted`: reflexive-transitive forward simulation.
- `sharedCore_stepStar_backward_restricted`: reflexive-transitive backward simulation.
- `sharedCore_stepStar_bisimulation`: combined star-level forward+backward shared-core bisimulation correspondence.

Thus, forward and backward simulation are now formalized for translated shared-core quote/drop/-parallel steps (and their star closure). Remaining future work is to extend this from shared-core to the full MeTTa fragment (including COMM/REFL) and prove an assumption-free cross-language bisimulation theorem.

5.6 Regression and Paper Map

`PaperMap.lean` (147 lines) provides a theorem-level index mapping each clause of the specification [7] to its Lean proof, including wrappers for step/star forward simulation, backward simulation, intrinsic backward inversion, and combined bisimulation (`paper_shared_to_rho_*` theorems). `CoreMain.lean` now also exposes an explicit HM endpoint map (`coreMain_hm_endpoint_recommendation_map`) distinguishing global canonical wrappers (assumption-scoped predecessor-finiteness) from the assumption-free scoped SC-quotiented family. `Regression.lean` (65 lines) provides a regression corpus with positive and negative test cases for both COMM and REFL rules. `RelationNames.lean` (28 lines) is a single source of truth for relation and builtin string identifiers.

6 Modal Mu-Calculus and the Rho-Mu Bridge

The modal μ -calculus [5, 1] extends Hennessy-Milner logic with least and greatest fixed-point operators. Our formalization covers standard semantics (`ModalMuCalculus.lean`, 360 lines).

6.1 Syntax

Formulas use de Bruijn indices for bound variables:

$$\phi ::= \top \mid \perp \mid \neg\phi \mid \phi \wedge \psi \mid \phi \vee \psi \mid \langle a \rangle \phi \mid [a] \phi \mid \mu X. \phi \mid \nu X. \phi \mid X$$

where $\langle a \rangle$ is diamond (possibility), $[a]$ is box (necessity), μX is least fixed point, and νX is greatest fixed point.

6.2 Rho-Calculus as LTS

The ρ -calculus reduction relation $p \rightsquigarrow q$ is a labelled transition system (LTS) with states = patterns, actions = $()$ (unlabeled), and transitions = $\rightsquigarrow$. This directly induces modal operators:

Definition 6.1 (Reduction-Based Modalities).

$$\begin{aligned} \text{possiblyProp } \phi p &:= \exists q. p \rightsquigarrow q \wedge \phi q \quad (\text{diamond}) \\ \text{relyProp } \phi p &:= \forall q. q \rightsquigarrow p \rightarrow \phi q \quad (\text{box}) \end{aligned}$$

Theorem 6.2 (Galois Connection IS Modal Duality).

$$(\forall p. \text{possiblyProp } \phi p \rightarrow \psi p) \iff (\forall p. \phi p \rightarrow \text{relyProp } \psi p)$$

This is exactly $\langle \rangle \phi \subseteq \psi \iff \phi \subseteq [] \psi$.

The proof is by direct unfolding of definitions (`galois_connection` in `Reduction.lean`). This validates the OSLF construction [10, 15]: operational semantics naturally gives rise to modal logic.

6.3 Spice Calculus as Iterated Diamond

The Spice lookahead corresponds precisely to iterated modal operators:

$$\begin{aligned} \text{presentMoment}(p) &\equiv \langle \rangle \phi \quad (1\text{-step}) \\ \text{futureStates}(p, n) &\equiv \langle \rangle^n \phi \quad (n\text{-step}) \\ \text{reachableViaStarClosure}(p) &\equiv \nu X. \phi \vee \langle \rangle X \quad (\text{eventually}) \end{aligned}$$

All three equivalences hold by definitional equality (`simp`). Precognitive agents computing Spice lookahead are computing μ -calculus temporal properties.

6.4 Conjecture: Rho Strictly More Expressive Than Mu

Conjecture 6.3 (ρ -Calculus Strictly More Expressive). *There exists a property expressible in ρ -calculus that is not expressible in modal μ -calculus. Specifically: μ -calculus cannot express “this process has the capability to quote itself.”*

The intuition is that μ -calculus formulas are determined entirely by LTS behavior (which transitions occur), while ρ -calculus processes can inspect their own syntactic structure via quote/unquote. Two processes $x[x]$ (self-quoting) and $y[z]$ (not self-quoting) are behaviorally indistinguishable—both are dead—yet differ structurally. No μ -formula should be able to distinguish them.

A proper formalization would require: (1) a coinductive bisimulation relation (not just deadlock equivalence), (2) a simulation theorem showing ρ can faithfully encode μ -calculus evaluation, and (3) the separation argument over the actual ρ -calculus types from `ProcessCalculi/RhoCalculus/`. This remains future work.

Remark 6.4 (Gödel Analogy). This is analogous to Gödel’s incompleteness theorem: first-order logic cannot express “this formula is provable”; adding a provability predicate (reflection) gives strictly more expressive power. Similarly, μ -calculus cannot express “this process quotes itself”; ρ -calculus can, via the quote operator `@`.

7 Quantale-Valued Semantics

Standard μ -calculus has Boolean semantics. We generalize to *quantale-valued* semantics [5] where satisfaction takes values in a commutative quantale Q (`ModalQuantaleSemantics.lean`, 520 lines).

Definition 7.1 (QLTS). A *quantale-labeled transition system* is a tuple $(S, A, \text{trans} : S \times A \times S \rightarrow Q)$ where Q is a commutative quantale with residuation.

Definition 7.2 (Quantale-Valued Satisfaction).

$$\begin{aligned} \text{qSatisfies}(s, \phi \wedge \psi) &= \text{qSatisfies}(s, \phi) \sqcap \text{qSatisfies}(s, \psi) \\ \text{qSatisfies}(s, \phi \vee \psi) &= \text{qSatisfies}(s, \phi) \sqcup \text{qSatisfies}(s, \psi) \\ \text{qSatisfies}(s, \langle a \rangle \phi) &= \bigsqcup_{s'} \text{trans}(s, a, s') \otimes \text{qSatisfies}(s', \phi) \\ \text{qSatisfies}(s, [a] \phi) &= \bigsqcap_{s'} \text{trans}(s, a, s') \Rightarrow \text{qSatisfies}(s', \phi) \\ \text{qSatisfies}(s, \mu X. \phi) &= \bigsqcap \{P \mid \text{transformer}(P) \sqsubseteq P\} \\ \text{qSatisfies}(s, \nu X. \phi) &= \bigsqcup \{P \mid P \sqsubseteq \text{transformer}(P)\} \end{aligned}$$

where $a \Rightarrow b := \bigsqcup \{z \mid z \otimes a \sqsubseteq b\}$ is left residuation.

Theorem 7.3 (Environment Monotonicity with Polarity). For formula ϕ and variable i with polarity $p \in \{\text{true}, \text{false}\}$: if $\rho_1(j) = \rho_2(j)$ for all $j \neq i$ and $\rho_1(i) \sqsubseteq \rho_2(i)$, then

$$\begin{aligned} p = \text{true} &\implies \text{qSatisfies}(\rho_1, \phi) \sqsubseteq \text{qSatisfies}(\rho_2, \phi) \\ p = \text{false} &\implies \text{qSatisfies}(\rho_2, \phi) \sqsubseteq \text{qSatisfies}(\rho_1, \phi) \end{aligned}$$

The proof (130 lines) proceeds by structural induction on ϕ , handling each constructor: base cases $(\top, \perp, X)$ are immediate; negation flips polarity via antitonicity of residuation; conjunction/disjunction preserve polarity via $\sqcap/\sqcup$ monotonicity; diamond/box use multiplication and residuation monotonicity; and the fixed-point cases (μ/ν) shift the de Bruijn variable index, extend the environment, and show every pre/post-fixed point for ρ_2 is also one for ρ_1 .

This enables the Knaster-Tarski theorem, proving that fixed points exist and are computable as limits of approximation sequences (`muApprox_mono`, `nuApprox_antimono`).

Example 7.4 (Instantiations). • **Probabilistic:** $Q = [0, 1]$ with standard multiplication.

- **PLN evidence:** $Q = \mathbb{R}_{\geq 0}^{\infty} \times \mathbb{R}_{\leq 0}^{\infty}$ (positive/negative evidence counts).
- **Fuzzy:** $Q = [0, 1]^n$ (multi-valued truth).
- **Tropical:** $Q = \mathbb{R}_{\geq 0}^{\infty}$ with $\min/+$ operations.

8 Common Infrastructure

The three process calculi share approximately 40% of their reduction and congruence infrastructure. A shared library in `ProcessCalculi/Common/` (4 files, 246 lines) eliminates this duplication through typeclasses and generic lemmas.

8.1 Typeclasses

`Common/ProcessAlgebra.lean` (31 lines) defines:

Listing 12: Shared process algebra typeclasses

```
class HasPar (P : Type) where
  par : P -> P -> P

class HasNu (P : Type) where
  nu : P -> P

class HasNil (P : Type) where
  nil : P
```

`Common/Congruence.lean` (64 lines) adds `HasSC` for structural congruence and the generic pattern `reduces_modulo_sc`.

8.2 Reflexive-Transitive Closure

`Common/Star.lean` (148 lines) provides two flavors of reflexive-transitive closure:

- **RTClosure:** Type-valued (carries witness data).
- **RTClosureProp:** Prop-valued (for logical reasoning).

Generic lifters—`liftUnary`, `liftBinLeft`, `liftBinRight`—allow multi-step reduction to be lifted through any unary or binary context constructor.

8.3 Instance Registration

Each calculus registers instances:

- **π -calculus:** `HasPar`, `HasNil`, `HasSC` (wrapping the Type-valued SC in `Nonempty`).
- **ρ -calculus:** `HasSC` (Prop-valued SC directly).
- **MQ-calculus:** `HasPar`, `HasNil`, `HasNu`, `HasSC` (all four typeclasses).

All three share: structural congruence as an equivalence relation with parallel commutativity/associativity and scope extrusion (unified via `HasSC`); and the COMM rule shape (sender + receiver $\rightarrow$ result), differing only in payload consumption (substitution in π/ρ , measurement in MQ).

9 Summary and Future Work

9.1 Summary

We have presented a comprehensive Lean 4.28 formalization of five process-algebraic systems and three cross-calculus bridges, totaling 17,821 lines with zero `sorry` and zero custom axioms (Table 1).

Key results include:

1. The Galois connection in ρ -calculus *is* modal diamond-box duality (Theorem 6.2), validating the OSLF construction.
2. Weak operational correspondence for restriction-free π -processes (`calculus_weak_correspondence_full_encoded` forward simulation (Theorem 3.1), backward admin reflection with three-branch decomposition (reflexive, admin-canonical, or reflected π -step), and their composition into a single theorem—under restriction-freeness and Barendregt hygiene hypotheses, totaling 2,745 lines.
3. Constructive both-outcomes witness for MQ measurement branching, with backend-parametric denotational semantics and post-collapse normalization.
4. We conjecture that ρ -calculus is strictly more expressive than μ -calculus (Conjecture 6.3): reflection cannot be captured by behavioral observation.
5. Complete quantale-valued μ -calculus semantics with environment monotonicity (Theorem 7.3), enabling Knaster-Tarski fixed-point computation across probabilistic, fuzzy, tropical, and PLN-evidence domains.
6. Spice calculus n -step lookahead is exactly iterated diamond $\langle \rangle^n$, proving that precognitive agents compute μ -calculus temporal properties.
7. Classical Hennessy-Milner equivalence (observational indistinguishability iff bisimilarity) proved assumption-free for finite-carrier SC-quotiented canonical ρ -relations, and under predecessor-finiteness for the global relation. Forward image-finiteness is auto-discharged for all `LanguageDef`-based relations. A structural sufficient condition for predecessor-finiteness (`LangPredFiniteSafe` [15]) identifies exactly which languages admit the global HM iff without quotienting; the ρ -calculus is provably excluded.
8. MeTTa-calculus with executable COMM/REFL reduction, premise-IR adequacy bridge, and a shared-fragment translation into ρ -calculus with forward/backward shared-core simulation and bisimulation at both step and step* levels (Section 5).

9.2 Future Work

1. **Unrestricted π -to- ρ bisimulation:** extend the weak correspondence from restriction-free processes to full π -calculus (including ν and replication), which requires encoding name-server apparatus into the theorem statement. Remove Barendregt side conditions via explicit alpha-renaming. Package a single assumption-minimal weak bisimulation iff.
2. **Global Hennessy-Milner for ρ -calculus:** lift the finite-carrier SC-quotiented HM iff to the global unquotiented ρ -relation. A decidable sufficient condition for predecessor-finiteness (`LangPredFiniteSafe` [15]) now provides the positive direction, but the ρ -calculus provably fails it (COMM erases the channel name). The remaining path is quotienting to a finite-image semantics or proving an alternative finiteness argument that accounts for name erasure.

3. **Full MeTTa $\leftrightarrow\rho$ bisimulation:** extend the shared-core (quote/drop/parallel) forward/backward simulation to COMM and REFL fragments; remove the source-anchored restriction by proving translation injectivity.
4. **Multi-qubit MQ gates:** extend gate semantics beyond single-wire action; prove collapse preservation under composition and measurement idempotence.
5. **PLN temporal bridge:** prove that PLN temporal operators correspond to modal formulas over a time-indexed LTS.
6. **OSLF integration for MQ:** instantiate MQ processes as an OSLF language instance with $\Diamond \dashv \Box$ Galois connection.
7. **Spice-MQ bridge:** connect ρ -calculus Spice n -step lookahead to MQ measurement prediction.

References

- [1] Julian Bradfield and Colin Stirling. Modal μ -calculi. In Patrick Blackburn, Johan van Benthem, and Frank Wolter, editors, *Handbook of Modal Logic*, pages 721–756. Elsevier, 2007.
- [2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [3] Simon J. Gay and Rajagopal Nagarajan. Communicating quantum processes. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 145–157, 2005.
- [4] André Joyal, Mogens Nielsen, and Glynn Winskel. Bisimulation from open maps. *Information and Computation*, 127(2):164–185, 1996.
- [5] Dexter Kozen. Results on the propositional μ -calculus. *Theoretical Computer Science*, 27:333–354, 1983.
- [6] Rasmus Kjær Lybech. *Encodability and Separation for a Reflective Higher-Order Calculus*. PhD thesis, IT University of Copenhagen, 2022. PhD Thesis.
- [7] L. Gregory Meredith. A symmetric reflective higher order concurrent calculus with backchaining. Unpublished specification, 2024.
- [8] L. Gregory Meredith. How the agents got their present moment. Preprint, 2026.
- [9] L. Gregory Meredith and David F. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.
- [10] L. Gregory Meredith and Mike Stay. Operational semantics in logical form. 2020. Draft / technical report. See also Williams & Stay, “Native Type Theory,” [arXiv:2102.04672](https://arxiv.org/abs/2102.04672).
- [11] Robin Milner. *Communicating and Mobile Systems: the π -Calculus*. Cambridge University Press, Cambridge, 1999.
- [12] Davide Sangiorgi and David Walker. *The π -Calculus: A Theory of Mobile Processes*. Cambridge University Press, 2001.

- [13] Mike Stay and L. Gregory Meredith. The MQ-calculus: A calculus for quantum processes involving intermittent measurement. Working draft, 2026.
- [14] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020.
- [15] Zar and Oruži (Claude Anthropic). Verified operational semantics in logical form: A Lean 4 formalization of the OSLF algorithm. Mettapedia Technical Report, 2025. leanOSLF.pdf.